



HikCentral Professional Video SDK (C++)

Developer Guide

Legal Information

© 2021 Hangzhou Hikvision Digital Technology Co., Ltd. All rights reserved.

This Document (hereinafter referred to be "the Document") is the property of Hangzhou Hikvision Digital Technology Co., Ltd. or its affiliates (hereinafter referred to as "Hikvision"), and it cannot be reproduced, changed, translated, or distributed, partially or wholly, by any means, without the prior written permission of Hikvision. Unless otherwise expressly stated herein, Hikvision does not make any warranties, guarantees or representations, express or implied, regarding to the Document, any information contained herein.

LEGAL DISCLAIMER

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE DOCUMENT IS PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". HIKVISION MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT. IN NO EVENT WILL HIKVISION BE LIABLE FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, IN CONNECTION WITH THE USE OF THE DOCUMENT, EVEN IF HIKVISION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.

Contents

Chapter 1 Overview	1
1.1 Introduction	1
1.2 Integrated Development Environment	1
1.3 Update History	1
1.4 Notices	1
Chapter 2 Typical Applications	2
2.1 Start Live View	2
2.2 Start Playback	4
2.3 Download Video Files	6
2.4 Start Two-Way Audio	8
Chapter 3 API Reference	11
3.1 Basic	11
3.1.1 Video_Init	11
3.1.2 Video_Fini	11
3.1.3 Video_GetLastError	11
3.2 Live View and Recording	12
3.2.1 Video_StartPreview	12
3.2.2 Video_StopPreview	14
3.2.3 Video_StartRecord	14
3.2.4 Video_StopRecord	15
3.3 Video Search	15
3.3.1 Video_GetVideoInfo	15
3.4 Playback and Downloading	16
3.4.1 Video_GetCurrentPlayTime	16
3.4.2 Video_StartPlayback	17
3.4.3 Video_StartPlayback_v2	19

3.4.4 Video_StartSingleFrame	22
3.4.5 Video_StopPlayback	22
3.4.6 Video_PlayCtrl	23
3.4.7 Video_StartDownload	24
3.4.8 Video_StartDownload_v2	25
3.4.9 Video_StopDownload	26
3.5 Capture and Audio Control	26
3.5.1 Video_PlaySnap	26
3.5.2 Video_GetVolume	27
3.5.3 Video_SetVolume	27
3.5.4 Video_SoundCtrl	28
3.6 Two-Way Audio	29
3.6.1 Video_StartTalk	29
3.6.2 Video_StopTalk	30
Chapter 4 Callback Function	31
4.1 pfnDecodedDataCallback	31
4.2 pfnDownloadCallback	32
4.3 pfnMsgCallback	32
4.4 pfnStreamCallback	33
Chapter 5 Data Structure	34
5.1 VIDEO_DETIALINFO	34
5.2 PVIDEO_DOWNLOADPARAM	34
5.3 PVIDEO_DOWNLOADPARAMEX	35
5.4 VIDEO_PLAYPRARM	35
5.5 VIDEORECORDPARAM	36
Appendix A. Error Code	37

Chapter 1 Overview

1.1 Introduction

The HikCentral Professional Video SDK based on C++ language provides several APIs to configure parameters for streaming, decoding, live view, recording, playback, video downloading, capturing, two-way audio, and so on.

1.2 Integrated Development Environment

Operating System

Microsoft® Windows 7 (32/64-bit)/Windows 8 (32/64-bit)/Windows 10 (32/64-bit)

Development Environment

Visual Studio 2015

1.3 Update History

Summary of Changes in Version 3.0.0_Oct., 2023

1. Added the functions of playback control, which support multiple time zone representations.
2. Added the functions of download control, which support multiple time zone representations.

Summary of Changes in Version 1.0.1_June., 2020

1. Added the functions of live view and getting stream.
2. Added the functions of playback and getting stream.
3. Added the functions of playback control, searching for playback time, capture during live view and playback, and audio control.
4. Added the functions of video downloading, checking video details (resolution, frame rate, encoding type, and container format), video downloading control, reverse playback, and two-way audio.

Summary of Changes in Version 1.0.1_Sept., 2019

New document.

1.4 Notices

Due to the limitations of Windows operating system and playing library, the size of 32-bit process memory must be smaller than 1.2 GB, and the number of ultra HD video decoding channels may be limited.

Chapter 2 Typical Applications

2.1 Start Live View

Live view helps to remotely get the real-time video or audio of monitoring area and know the status of human, vehicle, objects, and so on.

Steps

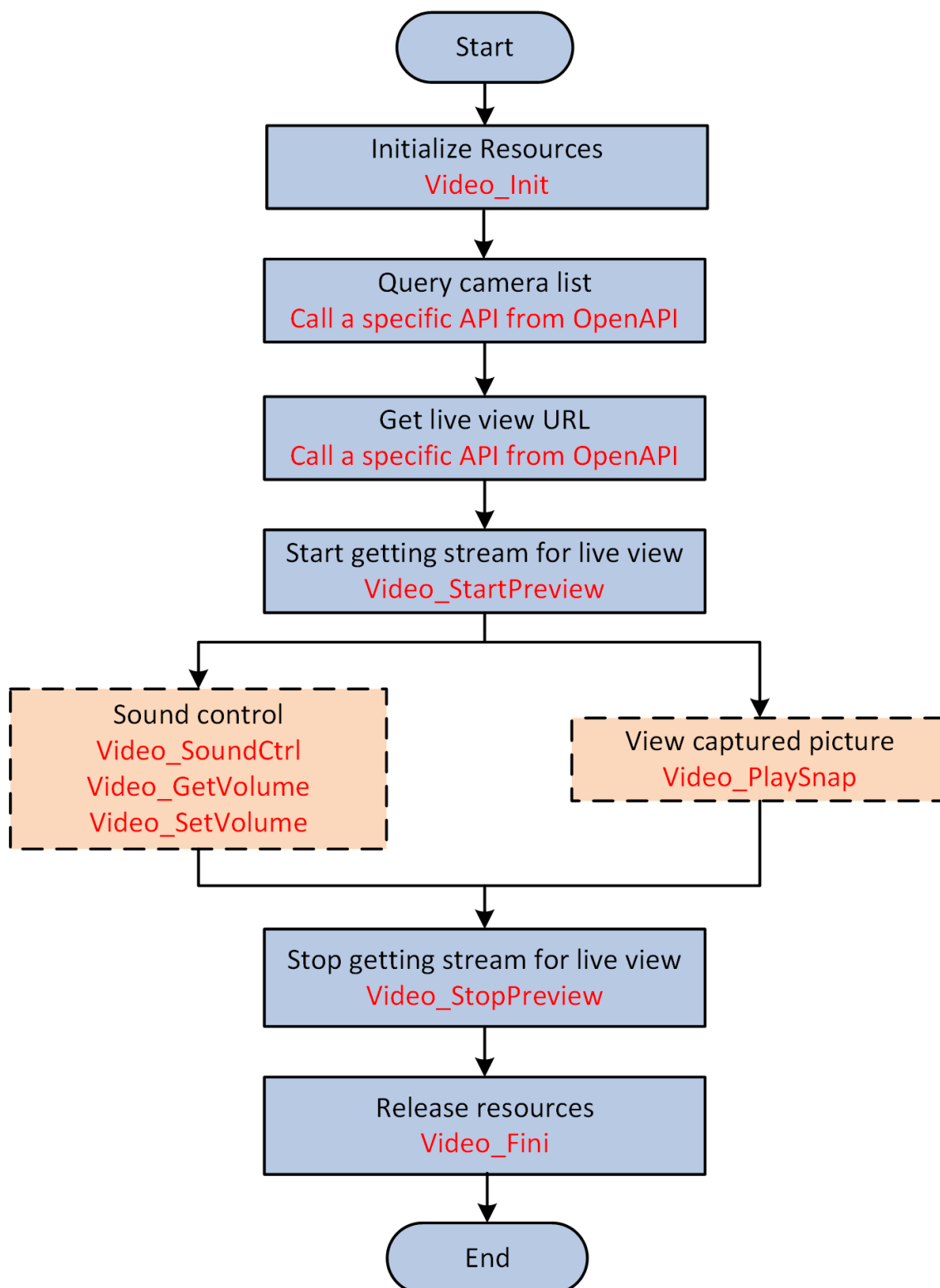


Figure 2-1 Programming Flow of Starting Live View

1. Call **Video_Init** to initialize the development environment.
2. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/resource/v1/camera/advance/cameraList** of OpenAPI to search for cameras and get camera information.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

3. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/video/v1/cameras/previewURLs** of OpenAPI to get the live view URL.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

4. Call **Video_StartPreview** to start live view.
5. **Optional:** Perform the following operation(s) after starting live view.

Turn on Audio Call **Video_SoundCtrl** .

Get Volume Call **Video_GetVolume** .

Set Volume Call **Video_SetVolume** .

Capture Picture Call **Video_PlaySnap** .

6. Call **Video_StopPreview** to stop live view.
7. Call **Video_Fini** to release resources.

2.2 Start Playback

After recording the videos, you can search for the video files and play the videos remotely.

Steps

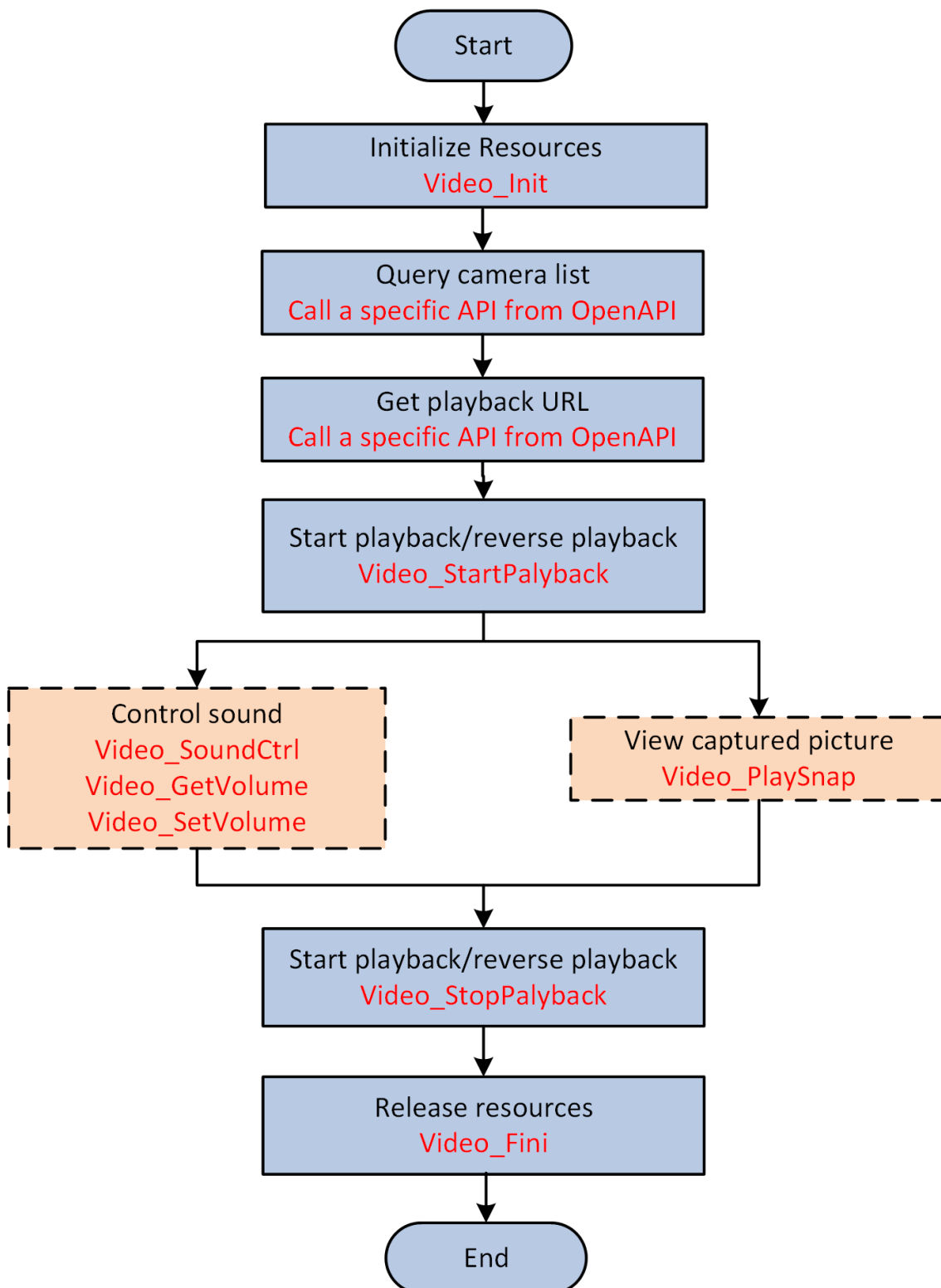


Figure 2-2 Programming Flow of Starting Playback

1. Call **Video_Init** to initialize the development environment.
2. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/resource/v1/camera/advance/cameraList** of OpenAPI to search for cameras and get camera information.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

3. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/video/v1/cameras/playbackURLs** of OpenAPI to get the playback URL.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

4. Call **Video_StartPlayback** to start playback or reverse playback.
5. **Optional:** Perform the following operation(s) after starting playback or reverse playback.

Turn on Audio Call **Video_SoundCtrl** .

Get Volume Call **Video_GetVolume** .

Set Volume Call **Video_SetVolume**

Capture Picture Call **Video_PlaySnap**

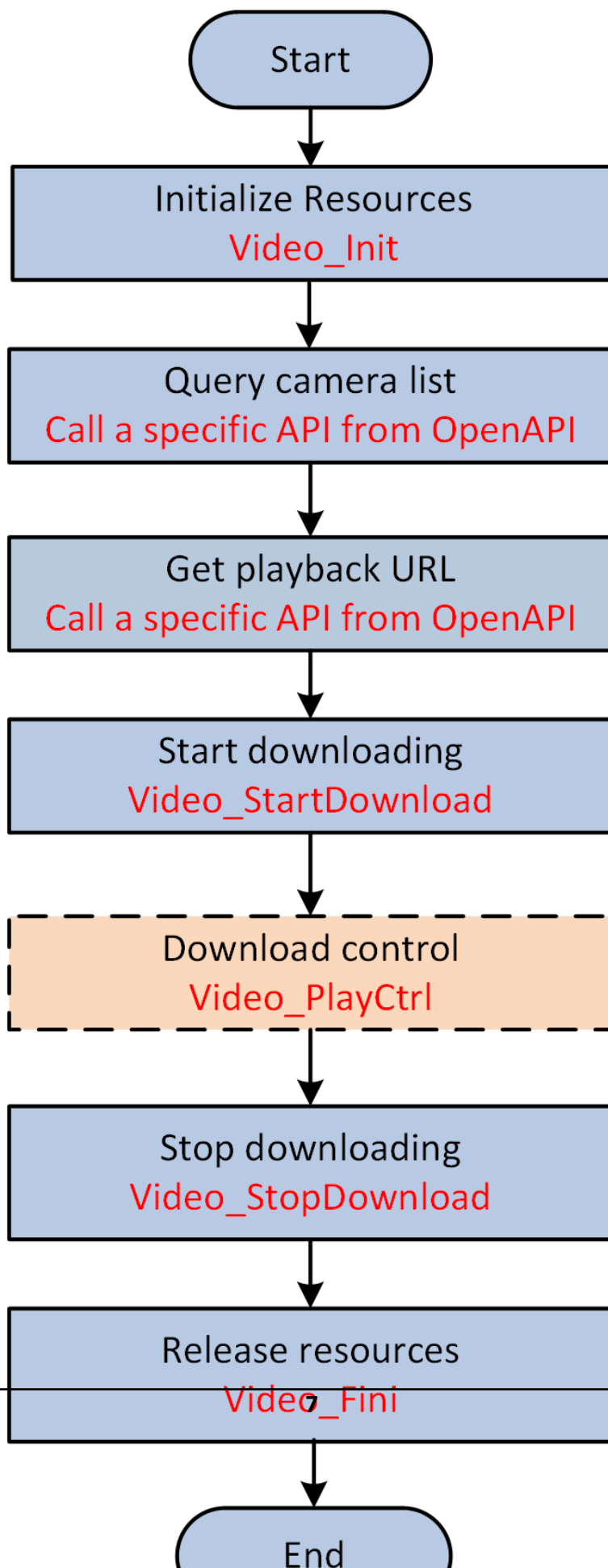
Control Playback Call **Video_PlayCtrl**

6. Call **Video_StopPlayback** to stop playback or reverse playback.
7. Call **Video_Fini** to release resources.

2.3 Download Video Files

You can download the video files stored in the device to the local computer for other processes.

Steps



1. Call **Video_Init** to initialize the development environment.
2. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/resource/v1/camera/advance/cameraList** of OpenAPI to search for cameras and get camera information.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

3. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/video/v1/cameras/playbackURLs** of OpenAPI to get the playback URL.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

4. Call **Video_StartDownload** to start downloading the video.
5. **Optional:** Call **Video_PlayCtrl** to control the downloading.
6. Call **Video_StopDownload** to stop downloading.
7. Call **Video_Fini** to release resources.

2.4 Start Two-Way Audio

The two-way audio provides the functions of voice talk between the client and specific cameras.

Steps

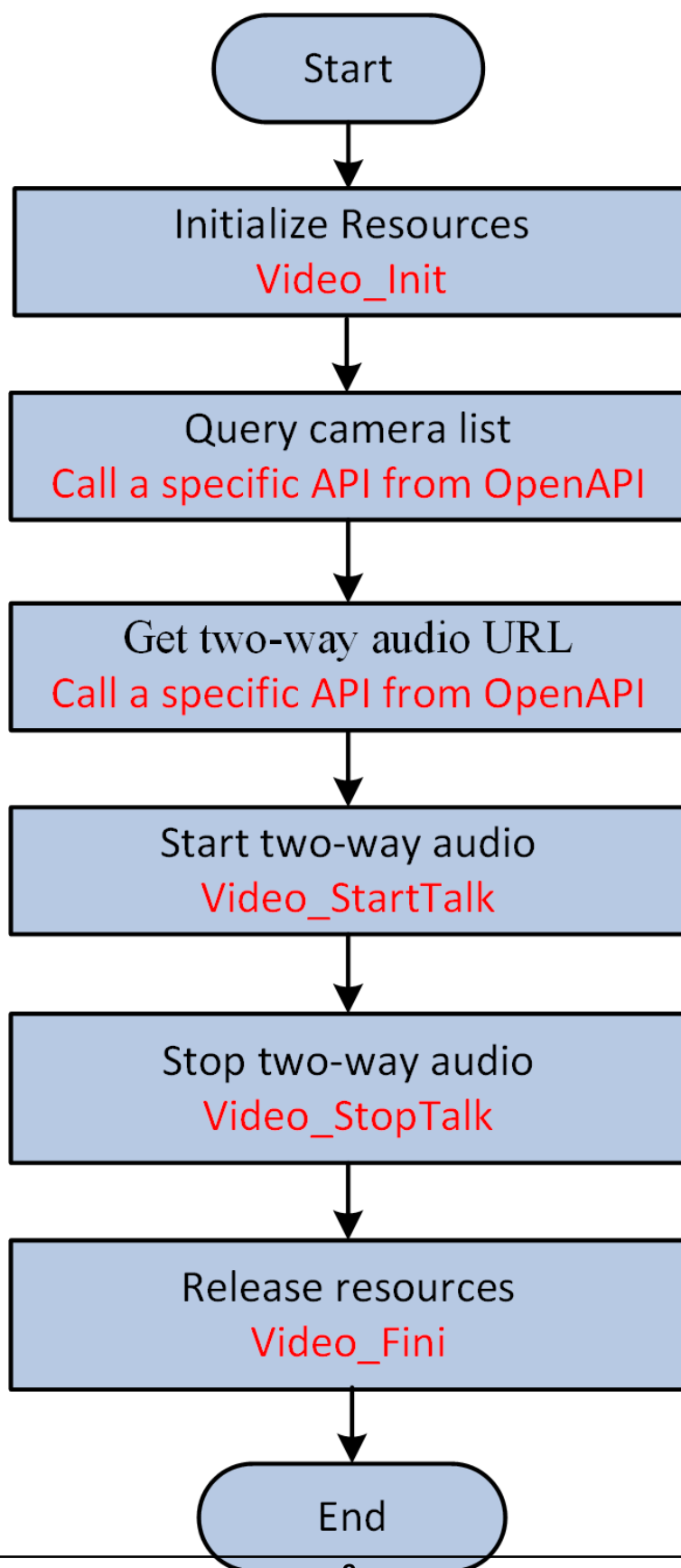


Figure 2-4 Programming Flow of Starting Two-Way Audio

1. Call **Video_Init** to initialize the development environment.
2. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/resource/v1/camera/advance/cameraList** of OpenAPI to search for cameras and get camera information.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

3. Call the API **POST https://[serverAddress]:[serverPort]/artemis/api/video/v1/cameras/talkURLs** of OpenAPI to get the two-way audio URL.



Note

Refer to *HikCentral Professional OpenAPI Developer Guide* for the API definition.

4. Call **Video_StartTalk** to start two-way audio.
5. Call **Video_StopTalk** to stop two-way audio.
6. Call **Video_Fini** to release resources.

Chapter 3 API Reference

3.1 Basic

3.1.1 Video_Init

Initialize the development environment before calling other APIs.

API Definition

```
int Video_Init(  
    const char    *pszEnv  
)
```

Parameters

pszEnv

[IN] Reserved, set to NULL.

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.1.2 Video_Fini

Release resources.

API Definition

```
int Video_Fini(  
)
```

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.1.3 Video_GetLastError

Get error codes.

API Definition

```
int Video_GetLastError(
)
```

Return Value

Return **Error Code** .

3.2 Live View and Recording

3.2.1 Video_StartPreview

Start getting stream for live view.

API Definition

```
int Video_StartPreview(
    const char      *pszUrl,
    void            *hWnd,
    PVIDEO_PLAYPRARM pstPlayReq
)
```

Parameters

pszUrl

[IN] Live view URL, which can be obtained via the API Gateway. Before starting live view, you should get the live view URL.

hWnd

[IN] Window handle

pstPlayReq

[IN] Request parameters of playing or getting stream. See **VIDEO_PLAYPRARM** for details.

Return Value

- Return -1 for failure, and return other value as the live view handle.
- If -1 is returned, you can call **Video_GetLastError** to get the error code.

Remarks

- For devices that only support main stream, you should search for the URL with main stream.
- The supported container format of stream is PS, RTP.
- You can call this API to set parameters of live view, stream callback function, YUV data callback function, and so on. See details in the table below:

hWnd	fnStream	fnMsg	fnDecodedStream	Function
Window handle	NULL	NULL	NULL	Live view
Window handle	Callback address	NULL	NULL	Live view, stream callback
Window handle	NULL	Callback address	NULL	Live view, streaming or playing status callback
Window handle	NULL	NULL	Callback address	Live view, YUV data callback
Window handle	Callback address	Callback address	NULL	Live view, stream callback, streaming or playing status callback
Window handle	Callback address	NULL	Callback address	Live view, stream callback, YUV data callback
Window handle	NULL	Callback address	Callback address	Live view, streaming status callback, YUV data callback
Window handle	Callback address	Callback address	Callback address	Live view, stream callback, streaming or playing status callback, YUV data callback
NULL	NULL	Callback address	NULL	Not support
NULL	NULL	NULL	Callback address	YUV data callback
NULL	Callback address	NULL	NULL	Stream callback
NULL	NULL	NULL	NULL	Not support
NULL	NULL	Callback address	Callback address	Streaming status callback, YUV data callback
NULL	Callback address	NULL	Callback address	Streaming status callback, YUV data callback

hWnd	fnStream	fnMsg	fnDecodedStream	Function
NULL	Callback address	Callback address	NULL	Stream callback, streaming status callback
NULL	Callback address	Callback address	Callback address	Stream callback, streaming status callback, YUV data callback

3.2.2 Video_StopPreview

Stop live view.

API Definition

```
int Video_StopPreview(
    int      iPlayHandle
)
```

Parameters

iPlayHandle

[IN] Live view handle, which is returned by [Video_StartPreview](#) .

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.2.3 Video_StartRecord

Start recording video during live view and save the video file to the local computer.

API Definition

```
int Video_StartRecord(
    int      iPlayHandle,
    const char *pszFileName,
    PVIDEORECORDPARAM pRecordParam
)
```

Parameters

iPlayHandle

[IN] Live view handle, which is returned by [Video_StartPreview](#) .

pszFileName

[IN] The absolute path (a string in ANSI encoding format) to save the videos in MP4 format. For example, *E:\test.mp4*.



Note

- Special characters * | ? are not allowed in the absolute path and file name.
 - Symbols "." and ".." are not recommended in the absolute path.
-

pstPlayReq

[IN] Recording parameters. See [VIDEORECORDPARAM](#) for details.

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.2.4 Video_StopRecord

Stop recording video during live view.

API Definition

```
int Video_StopRecord(  
    int      iPlayHandle  
)
```

Parameters

iPlayHandle

[IN] Live view handle, which is returned by [Video_StartPreview](#) .

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.3 Video Search

3.3.1 Video_GetVideoInfo

Search for video details, including resolution, frame rate, bit rate, encapsulation type, and encoding type.

API Definition

```
int Video_GetVideoInfo(  
    int          iPlayHandle,  
    VIDEO_DETIALINFO *stVideoDetailInfo  
)
```

Parameters

iPlayHandle

[IN] Live view or playback handle, which is returned by [Video_StartPreview](#) or [Video_StartPlayback](#).

stVideoDetailInfo

[OUT] Video details pointer. See [VIDEO_DETIALINFO](#) for details.

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.4 Playback and Downloading

3.4.1 Video_GetCurrentPlayTime

Search for the current playing timestamp during playback or reverse playback.

API Definition

```
int Video_GetCurrentPlayTime(  
    int          iPlayHandle  
)
```

Parameters

iPlayHandle

[IN] Playback handle, which is returned by [Video_StartPlayback](#).

Return Value

- Return playback timestamp for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.4.2 Video_StartPlayback

Start playback or reverse playback.

API Definition

```
int Video_StartPlayback(  
    const char        *pszUrl,  
    void              *hWnd,  
    int                iStartTimeStamp,  
    int                iEndTimeStamp,  
    PVIDEO_PLAYPRARM   pstPlayReq  
)
```

Parameters

pszUrl

[IN] Playback URL, which can be obtained via the API Gateway. Before starting playback, you should get the playback URL.



Note

For devices connected via ISUP, they can only get stream based on TCP.

hWnd

[IN] Window handle.

iStartTimeStamp

[IN] Playback start timestamp (UTC time), unit: second, e.g., 2019-09-27T09:00:00.

iEndTimeStamp

[IN] Playback end timestamp (UTC time), unit: second, e.g., 2019-09-27T09:40:55.

pstPlayReq

[IN] Request parameters of playing or streaming. See [**VIDEO_PLAYPRARM**](#) for details.

Return Value

- Return -1 for failure, and return other value as the playback handle.
- If -1 is returned, you can call [**Video_GetLastError**](#) to get the error code.

Remarks

- If the start time (**iStartTimeStamp**) is earlier than the end time (**iEndTimeStamp**), it will start normal playback; if the start time (**iStartTimeStamp**) is later than the end time

(**iEndTimeStamp**), reserve playback will start; if the start time and end time are same, error will occur.

- During normal playback, make sure the start time is later than the search start time of playback URL; during reverse playback, make sure the start time is earlier than the search end time of playback URL.
- The supported container format of stream is PS.
- You can call this API to set parameters of playback, reverse playback, stream callback function, YUV data callback function, and so on. See details in the table below:

hWnd	fnStream	fnMsg	fnDecodedStream	Function
Window handle	NULL	NULL	NULL	Playback or reverse playback
Window handle	Callback address	NULL	NULL	Playback or reverse playback, stream callback
Window handle	NULL	Callback address	NULL	Playback or reverse playback, streaming or playing status callback
Window handle	NULL	NULL	Callback address	Playback or reverse playback, YUV data callback
Window handle	Callback address	Callback address	NULL	Playback or reverse playback, stream callback, streaming or playing status callback
Window handle	Callback address	NULL	Callback address	Playback or reverse playback, stream callback, YUV data callback
Window handle	NULL	Callback address	Callback address	Playback or reverse playback, stream message callback, YUV data callback
Window handle	Callback address	Callback address	Callback address	Playback or reverse playback, stream callback, streaming or playing status callback, YUV data callback

hWnd	fnStream	fnMsg	fnDecodedStream	Function
NULL	NULL	Callback address	NULL	Not support
NULL	NULL	NULL	Callback address	YUV data callback
NULL	Callback address	NULL	NULL	Stream callback
NULL	NULL	NULL	NULL	Not support
NULL	NULL	Callback address	Callback address	Streaming status callback, YUV data callback
NULL	Callback address	NULL	Callback address	Stream callback, YUV data callback
NULL	Callback address	Callback address	NULL	Stream callback, streaming status callback
NULL	Callback address	Callback address	Callback address	Stream callback, streaming status callback, YUV data callback

3.4.3 Video_StartPlayback_v2

Start playback or reverse playback. Supports multiple time zone representations.

API Definition

```
int Video_StartPlayback_v2(
    const char      *pszUrl,
    void            *hWnd,
    const char      *StartTimeStamp,
    const char      *EndTimeStamp,
    PVIDEO_PLAYPRARM pstPlayReq
)
```

Parameters

pszUrl

[IN] Playback URL, which can be obtained via the API Gateway. Before starting playback, you should get the playback URL.



Note

For devices connected via ISUP, they can only get stream based on TCP.

hWnd

[IN] Window handle.

StartTimeStamp

[IN] Playback start timestamp (UTC time), unit: second, e.g., 2019-09-27T09:00:00.

EndTimeStamp

[IN] Playback end timestamp (UTC time), unit: second, e.g., 2019-09-27T09:40:55.

pstPlayReq

[IN] Request parameters of playing or streaming. See **VIDEO_PLAYPRARM** for details.

Return Value

- Return -1 for failure, and return other value as the playback handle.
- If -1 is returned, you can call **Video_GetLastError** to get the error code.

Remarks

- If the start time (**iStartTimeStamp**) is earlier than the end time (**iEndTimeStamp**), it will start normal playback; if the start time (**iStartTimeStamp**) is later than the end time (**iEndTimeStamp**), reserve playback will start; if the start time and end time are same, error will occur.
- During normal playback, make sure the start time is later than the search start time of playback URL; during reverse playback, make sure the start time is earlier than the search end time of playback URL.
- The supported container format of stream is PS.
- You can call this API to set parameters of playback, reverse playback, stream callback function, YUV data callback function, and so on. See details in the table below:

hWnd	fnStream	fnMsg	fnDecodedStream	Function
Window handle	NULL	NULL	NULL	Playback or reverse playback
Window handle	Callback address	NULL	NULL	Playback or reverse playback, stream callback
Window handle	NULL	Callback address	NULL	Playback or reverse playback, streaming or playing status callback

hWnd	fnStream	fnMsg	fnDecodedStream	Function
Window handle	NULL	NULL	Callback address	Playback or reverse playback, YUV data callback
Window handle	Callback address	Callback address	NULL	Playback or reverse playback, stream callback, streaming or playing status callback
Window handle	Callback address	NULL	Callback address	Playback or reverse playback, stream callback, YUV data callback
Window handle	NULL	Callback address	Callback address	Playback or reverse playback, stream message callback, YUV data callback
Window handle	Callback address	Callback address	Callback address	Playback or reverse playback, stream callback, streaming or playing status callback, YUV data callback
NULL	NULL	Callback address	NULL	Not support
NULL	NULL	NULL	Callback address	YUV data callback
NULL	Callback address	NULL	NULL	Stream callback
NULL	NULL	NULL	NULL	Not support
NULL	NULL	Callback address	Callback address	Streaming status callback, YUV data callback
NULL	Callback address	NULL	Callback address	Stream callback, YUV data callback

hWnd	fnStream	fnMsg	fnDecodedStream	Function
NULL	Callback address	Callback address	NULL	Stream callback, streaming status callback
NULL	Callback address	Callback address	Callback address	Stream callback, streaming status callback, YUV data callback

3.4.4 Video_StartSingleFrame

Start single frame playback.

API Definition

```
int Video_StartSingleFrame(
    int      iPlayHandle
)
```

Parameters

iPlayHandle

[IN] Playback handle, which is returned by [Video_StartPlayback](#) .

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.4.5 Video_StopPlayback

Stop playback and reverse playback.

API Definition

```
int Video_StopPlayback(
    int      iPlayHandle
)
```

Parameters

iPlayHandle

[IN] Playback handle, which is returned by [Video_StartPlayback](#) .

Return Value

- Return *0* for success, and return *-1* for failure.
- If *-1* is returned, you can call **Video_GetLastError** to get the error code.

3.4.6 Video_PlayCtrl

Control playing and video downloading.

API Definition

```
int Video_PlayCtrl(  
    int      iPlayHandle,  
    int      iCtrlType,  
    int      iParam  
)
```

Parameters

iPlayHandle

[IN] Playback handle, which is returned by **Video_StartPlayback** or **Video_StartDownload** .

iCtrlType

[IN] Control types: 0-pause playing, 1-resume playing, 2-located playing, 3-fast or slow forward playing, 4-pause downloading, 5-resume downloading.

iParam

[IN] Values vary with the control types (iCtrlType), see details in the table below:

iCtrlType	iParam	Remark
0	0	/
1	0	The playing speed will restore to the normal speed.
2	2	The timestamp needs to be between the start timestamp and the end timestamp, unit: second.
3	3	The following speed levels are supported: 16×, 8×, 4×, 2×, 1×, -2×, -4×, -8×, and -16×
4	0	/
5	0	/

Return Value

- Return *0* for success, and return *-1* for failure.
- If *-1* is returned, you can call **Video_GetLastError** to get the error code.

3.4.7 Video_StartDownload

Start downloading the video to the local computer.

API Definition

```
int Video_StartDownload(  
    const char          *pszUrl,  
    const char          *pszFileName,  
    PVIDEO_DOWNLOADPARAM pstDownloadReq  
)
```

Parameters

pszUrl

[IN] Playback URL, which can be obtained via the API Gateway. Before starting live view, you should get the live view URL.



Note

For devices connected via ISUP, they can only get stream based on TCP.

pszFileName

[IN] The absolute path (a string in ANSI encoding format) to save the videos in MP4 format. For example, *E:\test.mp4*.



Note

- Special characters * | ? are not allowed in the absolute path.
 - Symbols "." and ".." are not recommended in the absolute path.
 - If the format of file to be downloaded is specified as AVI (e.g., *E:\test.avi*), the file cannot be divided in to multiple packages during the downloading. And if the stream encoding standard of the downloaded AVI file is H.265, the file cannot be played via the normal player.
-

pstDownloadReq

[IN] Request parameters of video downloading. See [***PVIDEO_DOWNLOADPARAM***](#) for details.

Return Value

- Return playback or downloading handle for success, and return -1 for failure.
- If -1 is returned, you can call [***Video_GetLastError***](#) to get the error code.

Remarks

- Video files can only be downloaded to local computer.
- For the video files of Hikvision devices, you must play them via the Hikvision player.

- After downloading the videos, you should call **Video_StopDownload** to release the downloading resources.
- The supported video container formats are PS, RTP.

3.4.8 Video_StartDownload_v2

Start downloading the video to the local computer. Supports multiple time zone representations.

API Definition

```
int Video_StartDownload_2(  
    const char          *pszUrl,  
    const char          *pszFileName,  
    PVIDEO_DOWNLOADPARAMEX    pstDownloadReq  
)
```

Parameters

pszUrl

[IN] Playback URL, which can be obtained via the API Gateway. Before starting live view, you should get the live view URL.



Note

For devices connected via ISUP, they can only get stream based on TCP.

pszFileName

[IN] The absolute path (a string in ANSI encoding format) to save the videos in MP4 format. For example, **E:\test.mp4**.



Note

- Special characters * | ? are not allowed in the absolute path.
 - Symbols "." and ".." are not recommended in the absolute path.
 - If the format of file to be downloaded is specified as AVI (e.g., **E:\test.avi**), the file cannot be divided in to multiple packages during the downloading. And if the stream encoding standard of the downloaded AVI file is H.265, the file cannot be played via the normal player.
-

pstDownloadReq

[IN] Request parameters of video downloading. See **PVIDEO_DOWNLOADPARAM** for details.

Return Value

- Return playback or downloading handle for success, and return -1 for failure.
- If -1 is returned, you can call **Video_GetLastError** to get the error code.

Remarks

- Video files can only be downloaded to local computer.
- For the video files of Hikvision devices, you must play them via the Hikvision player.
- After downloading the videos, you should call **Video_StopDownload** to release the downloading resources.
- The supported video container formats are PS, RTP.

3.4.9 Video_StopDownload

Stop downloading videos.

API Definition

```
int Video_StopDownload(  
    int      iPlayHandle  
)
```

Parameters

iPlayHandle

[IN] Downloading handle, which is returned by **Video_StartDownload** .

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call **Video_GetLastError** to get the error code.

3.5 Capture and Audio Control

3.5.1 Video_PlaySnap

Capture pictures during live view, playback, or reverse playback.

API Definition

```
int Video_PlaySnap(  
    int      iPlayHandle,  
    const char *pszFileName  
)
```

Parameters

iPlayHandle

[IN] Live view or playback handle, which is returned by [Video_StartPreview](#) or [Video_StartPlayback](#) .

pszFileName

[IN] The absolute path (a string in ANSI encoding format) to save the pictures with different formats (i.e., BMP and JPG). For example, *E:/SnapShot/camera01.jpg*.



Note

- Special characters * | ? are not allowed in the saving path and the picture name.
 - Symbols "." and ".." are not recommended in the saving path.
-

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

3.5.2 Video_GetVolume

Get the volume, which is between 0 and 100.

API Definition

```
int Video_GetVolume(  
    int      iPlayHandle  
)
```

Parameters

iPlayHandle

[IN] Live view or playback handle, which is returned by [Video_StartPreview](#) or [Video_StartPlayback](#) .

Return Value

- Return -1 for failure, and return the volume value for success.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

Remarks

This function is valid only when the stream data contains audio data.

3.5.3 Video_SetVolume

Set the volume.

API Definition

```
int Video_SetVolume(  
    int      iPlayHandle,  
    int      iVolume  
)
```

Parameters

iPlayHandle

[IN] Live view or playback handle, which is returned by [Video_StartPreview](#) or [Video_StartPlayback](#).

iCtrlType

[IN] Volume, range: [0, 100].

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call [Video_GetLastError](#) to get the error code.

Remarks

This function is valid only when the stream data contains audio data.

3.5.4 Video_SoundCtrl

Turn on or off the audio during live view or playback.

API Definition

```
int Video_SoundCtrl(  
    int      iPlayHandle,  
    int      iCtrlType  
)
```

Parameters

iPlayHandle

[IN] Live view or playback handle, which is returned by [Video_StartPreview](#) or [Video_StartPlayback](#).

iCtrlType

[IN] Control types: 0-turn on audio, 1-turn off audio, other values-undefined.

Return Value

- Return 0 for success, and return -1 for failure.
- If -1 is returned, you can call **Video_GetLastError** to get the error code.

Remarks

This function is valid only when the stream data contains audio data.

3.6 Two-Way Audio

3.6.1 Video_StartTalk

Start two-way audio.

API Definition

```
int Video_StartTalk(  
    const char      *pszUrl,  
    const char      *pszFileName,  
    pfnMsgCallback  fnMsg,  
    void            *pUserData  
)
```

Parameters

pszUrl

[IN] Two-way audio URL, which can be obtained via OpenAPI. Before starting two-way audio, you should get the two-way audio URL.

pszFileName

[IN] The absolute path to save the audio file in WAV format. For example, **D:\test.wav**. You can set it to NULL if it is not required.



Note

- Special characters * | ? are not allowed in the absolute path.
 - Symbols "." and ".." are not recommended in the absolute path.
-

fnMsg

[IN] Get two-way audio messages via the callback function **pfnStreamCallback** . You can set it to NULL if it is not required.

pUserData

[IN] User data, which can be passed through by **pfnStreamCallback** . You can set it to NULL if it is not required.

Return Value

- Return *0* for success, and return *-1* for failure.
- If *-1* is returned, you can call **Video_GetLastError** to get the error code.

Remarks

- One client can only talk with one device at one time, and it does not support talking with the devices that connecting to the NVR.
- Before start two-way audio, make sure you have a recording device available on your computer.

3.6.2 Video_StopTalk

Stop two-way audio.

API Definition

```
int Video_StopTalk(  
)
```

Return Value

- Return *0* for success, and return *-1* for failure.
- If *-1* is returned, you can call **Video_GetLastError** to get the error code.

Chapter 4 Callback Function

4.1 pfnDecodedDataCallback

Callback function of YUV data

Callback Function Definition

```
void (__stdcall* pfnDecodeDataCallback)(
    int          iPlayHnadle,
    const char    *pDataArray,
    int          iDataLen,
    int          iWidth,
    int          iHeight,
    int          iFrameType,
    int          iTimeStamp,
    void         *pUserData
)
```

Parameters

iPlayHnadle

Live view or playback handle, which is returned by [Video_StartPreview](#) or [Video_StartPlayback](#).

pdataArray

Decoded data, which must be used as a binary array.

iDataLen

Data size.

iWidth

Image width.

iHeight

Image height.

iFrameType

YUV image type, currently, the value is 3, that is, the type is YV12.

iTimeStamp

Timestamp

pUserData

Custom data.

4.2 pfnDownloadCallback

Callback function of video downloading progress

Callback Function Definition

```
void (__stdcall* pfnDownloadCallback)(  
    int        iDownloadHandle,  
    float       fPercent,  
    int        iMsg,  
    void        *pUserData  
);
```

Parameters

iDownloadHandle

Video downloading handle.

fPercent

Video downloading progress

iMsg

Video downloading status: 0-start downloading, 1-downloading, 2-downloaded, 3-start packaging video, 4-packaging video failed, 5-video packaged, 6-streaming stopped during downloading.

pUserData

Custom data.

4.3 pfnMsgCallback

Callback function of streaming and playing status.

Callback Function Definition

```
void (__stdcall * pfnMsgCallback)(  
    int        iPlayHandle,  
    int        iMessage,  
    void        *pUserData  
);
```

Parameters

iPlayHandle

Live view or playback handle, which is returned by [**Video_StartPreview**](#) or [**Video_StartPlayback**](#).



Note

For two-way audio function, set **iPlayHnadle** to -1.

iMessage

Streaming and playing status: 1-start playing, 2-stop playing, 3-playing exception, 10-start getting stream, 11-stop getting stream, 12-streaming exception.

pUserData

Custom data.

4.4 pfnStreamCallback

Stream callback function

Callback Function Definition

```
void (__stdcall* pfnStreamCallback)(
    int      iPlayHnadle,
    int      iStreamDataType,
    const char *pDataArray,
    int      iDataLen,
    void     *pUserData
);
```

Parameters

iPlayHnadle

Live view or playback handle, which is returned by [***Video_StartPreview***](#) or [***Video_StartPlayback***](#).

iStreamDataType

Video data type: 0-header, 1-data, 2-end tag

pdataArray

Stream data, which must be used as a binary array.

iDataLen

Data size.

pUserData

Custom data.

Chapter 5 Data Structure

5.1 VIDEO_DETIALINFO

Video details structure

Member	Data Type	Description
iWidth	int	Image width.
iHeight	int	Image height.
iEncodeType	int	Encoding type: 1-H264, 2-MPEG2_PS, 3-MPEG4, 4-MJPEG, 5-H265, 6-SVAC
iEncapsulationType	int	Container format: 1-PS
iFrameRate	int	Frame rate.
szReserve	Array of char	Reserved.

5.2 PVIDEO_DOWNLOADPARAM

Structure about video downloading parameters

Member	Data Type	Description
fnDownload	<u><i>pfnDownloadCallback</i></u>	Downloading progress.
pUserData	void*	User data pointer.
iFileMaxSize	int	Package size of video segment.
iMaxRecordSize	int	Total size of video file.
iStartTimeStamp	int	Start timestamp of the video downloading, unit: second
iEndTimeStamp	int	End timestamp of the video downloading, unit: second
szReserve	Array of char	Reserved.

5.3 PVIDEO_DOWNLOADPARAMEX

Structure about video downloading parameters. Supports multiple time zone representations.

Member	Data Type	Description
fnDownload	<u><i>pfnDownloadCallback</i></u>	Downloading progress.
pUserData	void*	User data pointer.
iFileMaxSize	int	Package size of video segment.
iMaxRecordSize	int	Total size of video file.
szStartTimeStamp	Array of char	Start time of recording, represented by "YYYY-MM-DDTHH:MM:SS+current time zone", e.g., "2023-09-18T01:00:00+08:00".
szEndTimeStamp	Array of char	End time of recording, represented by "YYYY-MM-DDTHH:MM:SS+current time zone", e.g., "2023-09-18T01:00:00+08:00".
szReserve	Array of char	Reserved.

5.4 VIDEO_PLAYPRARM

Playing request structure

Parameter	Data Type	Description
iHardWareDecode	int	Whether to enable hardware decoding: 0-no, 1-yes, other values-parameter error.  Note • Hardware decoding should be supported by graphics card. • If failed to enable hardware decoding, it will automatically switch to software decoding. • Generally, it is not recommended to enable hardware decoding.
fnStream	<u><i>pfnStreamCallback</i></u>	Callback function of network video stream
fnMsg	<u><i>pfnMsgCallback</i></u>	Callback function of streaming and playback.

Parameter	Data Type	Description
fnDecodedStream	<u>pfnDecodedDataCallback</u>	Callback function of decoded YUV data.
pUserData	void*	User data pointer
szReserve	Array of char	Reserved, the maximum size is 64 bytes.

5.5 VIDEORECORDPARAM

Structure about the parameters of local video downloading

Parameter	Data Type	Description
fnDownload	<u>pfnDownloadCallback</u>	Callback function of downloading progress.
IFileMaxSize	int	Package size of video segment.
pUserData	void*	User data pointer

Appendix A. Error Code

The errors occurred during the SDK integration are listed here for reference. You can search for the error description according to the returned error code.

Error Name	Error Code	Description
VIDEO_ERR_SUCCESS	0	Succeeded.
VIDEO_ERR_PARAM	1	Incorrect parameter.
VIDEO_ERR_INVALID_POINTER	2	Invalid pointer.
VIDEO_ERR_NOT_INIT	3	Resource uninitialized.
VIDEO_ERR_INVALID_HANDLE	4	Invalid handle.
VIDEO_ERR_CREATE_INSTANCE	5	Creating handle failed.
VIDEO_ERR_APPLY_MEMORY	6	Applying resource failed.
VIDEO_ERR_NET_EXCEPTION	7	Network error.
VIDEO_ERR_FILE_PATH_SIZE	8	Invalid file path or the file path's size is larger than 1024 bytes.
VIDEO_ERR_OPENFILE	9	Opening file failed.
VIDEO_ERR_START_STREAM	100	Getting stream failed.
VIDEO_ERR_STOP_STREAM	101	Stopping getting stream exception.
VIDEO_ERR_STREAM_RATE	102	Setting streaming speed failed.
VIDEO_ERR_RESUME_STREAM	103	Resuming getting stream failed.
VIDEO_ERR_STREAMLIB_INIT	104	Initialize stream media server failed.
VIDE_ERR_STREAM_FORCEIFRAME	105	Forcing inserting I-frame failed.
VIDEO_ERR_TIMESTAMP	201	Invalid timestamp.
VIDEO_ERR_SOUND_OPEN	202	Turning on audio failed.
VIDEO_ERR_SOUND_CLOSE	203	Turning off audio failed.
VIDEO_ERR_SOUND_SETVOLUM	204	Setting svolume failed.
VIDEO_ERR_SOUND_GETVOLUM	205	Getting volume failed.
VIDEO_ERR_SNAP	206	Capture failed.
VIDEO_ERR_PLAYCTRL	207	Playing control failed.

Error Name	Error Code	Description
VIDEO_ERR_START_GPU_PLAY	208	Enabling hardware encoding failed.
VIDEO_ERR_PLAYCTRL_PORT	209	Getting playing port of PlayCtrl library failed.
VIDEO_ERR_PLAYCTRL_PLAY	210	Playing control failed.
VIDEO_ERR_POSITINO_TIMESTAMP_PLAY	211	Incorrect timestamp of located playing.
VIDEO_ERR_SINGLE_FRAME	212	Starting single frame playback failed.
VIDEO_ERR_TALK	301	Starting two-way audio failed.
VIDEO_ERR_DEVICE	302	Device error.
VIDEO_ERR_NOT_AUDIO_CARD	303	No sound card, or the type of sound card is not supported.
VIDEO_ERR_AUDIOCOM	304	Audio library error.
VIDEO_ERR_AUDIO_STREAM	305	Getting stream during two-way audio failed.
VIDEO_ERR_AUDIO_CREATECAPTUREHANDLE	306	Creating port No. of collection and encoding module failed.
VIDEO_ERR_AUDIO_REGISTCAPUTREDATA	307	Registring callback function of PCM audio data failed.
VIDEO_ERR_AUDIO_STARTCAPUTRE	308	Start collection and encoding failed.
VIDEO_ERR_AUDIO_STARTPLAY	309	Playing audio failed.
VIDEO_ERR_AUDIO_DECODEDATA	310	Decoding audio failed during two-way audio.
VIDEO_ERR_AUDIOCOM_UNINIT	311	Two-way audio deinitialization failed.
VIDEO_ERR_INNER	1000	Internal error.
VIDEO_ERR_OPER_ORDER	1001	Incorrect operation sequence.
VIDEO_ERR_DISKSPACE	1002	Insufficient disk space.
VIDEO_ERR_CREATEFILE	1004	Creating file failed.
VIDEO_ERR_HANDLE	1005	Invalid handle.
VIDEO_ERR_DISK_FULL	1006	Insufficient disk space.

Error Name	Error Code	Description
VIDEO_ERR_STOPDOWNLOAD	1007	Stopping downloading failed.
VIDEO_ERR_OTHER	10000	Other error.



See Far, Go Further